ПРИРОДО-МАТЕМАТИЧЕСКА ГИМНАЗИЯ
СЪС ЗАСИЛЕНО ИЗУЧАВАНЕ НА ЧУЖДИ ЕЗИЦИ
“ИВАН ВАЗОВ”-ДИМИТРОВГРАД

ДИПЛОМЕН ПРОЕКТ

Тема:

Приложение за отдаване коли под наем (с възможност за личен шофьор)

Дипломант: Александър Димитров

Ръководител: Тихомира Тенчева

Специалност - код: Приложно програмиране – 4810301

Професия – код: Приложен програмист – 481030

Димитровград, 2026

Съдържание

Увод ... 3
I.Въведение в индустрията на услугите за отдаване на автомобили под наем ... 3
II.Необходимостта от PremiumCarRentX ... 3
III.Основни предизвикателства пред управлението на Rent-a-car услуги ... 3
1. Сложност на администрирането ... 3
2. Трудност при съхраняването и управлението на данни ... 4
3. Липса на интеграция и автоматизация ... 4
IV.Как PremiumCarRentX помага за решаване на тези проблеми? ... 4
V.Значение на дигитализацията в съвременния бизнес ... 4
MVC (Model-View-Controller) ... 5
I. ПРЕДМЕТ И ЗАДАЧИ НА MVC ... 5
II. КОМПОНЕНТИ НА MVC ... 5
1. Модел (Model) ... 5
2. Изглед (View) ... 5
3. Контролер (Controller) ... 6
III.Предимства на MVC Архитектурата: ... 6
IV.Приложения на MVC: ... 6
Основна част (Структура на проекта) ... 7
I. Слой за база данни (DataLayer) ... 8
1. Entities (Примерът с клас Car) ... 8
2. Миграции в Entity Framework ... 9
3. DbContext (Пример с ApplicationDbContext) ... 9
II. СЛОЙ ЗА УСЛУГИ(SERVICE LAYER) ... 16
1.Роля на слоя на услугите ... 16
2.Структура на слой на услугите ... 16
III. СПОДЕЛЕН ПРОЕКТ(SHARED) ... 20
1.Attributes(Пример с AutoBindAttribute) ... 21
2.Dtos(Пример с MatchDto) ... 21
3.Enum(Пример с Positions) ... 22
4.Repos(Пример с IBaseRepository) ... 24

5.Services(Пример с IBaseCrudService) 26
IV. ПРЕЗЕНТАЦИОНЕН СЛОЙ(PRESENTATION LAYER) 28
1.Основни функции на презентационния слой в MVC: 28
2.Controllers(Пример с MatchController) 28
3.ViewModels(Пример с LoginVM) 32
4.Views(Пример с _Layout.cshtml и List на Match) 33
5.Appsettings 40
6.AutoMapper(Пример с AutoMapperConfiguration.cs) 41
7.Program.cs 45
V.КОМПОНЕНТНИ ТЕСТОВЕ(UNIT TESTS) 48
1.Tests(Пример с BaseRepositoryTests) 48
Програмна реализация 51
I.Използвани технологии 51
1.HTML 52
2.CSS 52
3.JavaScript 52
4.SQL 52
5.SSMS 53
Заключение 53
I.Начини за подобрение на FootballManagement 53
Придобити знания 55
Използвана литература 56
План за управление на риска 57
Приложения 58
I.Use case diagram 58
II.Swot table 59
III.Работещ проект 59

Увод

I. Въведение в индустрията на услугите за отдаване на автомобили под наем

Секторът на услугите за отдаване на автомобили под наем (Rent-a-car) е един от най-динамичните компоненти на съвременната транспортна индустрия, осигуряващ мобилност на милиони хора по света. Управлението на един автопарк не се ограничава само до поддръжката на превозните средства – то включва сложни административни, логистични и стратегически процеси, които изискват изключителна прецизност и ефективност. От управлението на наличните автомобили и координацията с личните шофьори до прецизното планиране на графиците за наемане – всички тези аспекти са критични за успеха и репутацията на компанията.

Въпреки че в миналото са използвани традиционни методи за управление, базирани на хартиени договори и обемни електронни таблици, те вече не отговарят на високите изисквания за бързина и сигурност в съвременния свят. С навлизането на новите технологии и масовата дигитализация се появява остра необходимост от интегрирани софтуерни решения, които да автоматизират процесите, да намалят човешките грешки и да осигурят незабавен достъп до важна информация в реално време.

II. Необходимостта от PremiumCarRentX

В този контекст е разработено приложението **PremiumCarRentX** – съвременна платформа за управление на услуги за отдаване на автомобили под наем, която предлага професионално решение за администриране на ключовите аспекти на този бизнес. Приложението е предназначено както за големи автопаркове, така и за бутикови Rent-a-car агенции, които се стремят към оптимизация на дейността си и постигане на високи финансови резултати.

PremiumCarRentX предоставя цялостна екосистема за управление на автомобили, профили на професионални шофьори и потребителски резервации. Със своя иновативен интерфейс и мощни функционалности, системата улеснява работата на администраторите и подобрява клиентското изживяване. Внедряването на платформата не само

минимизира времето за обработка на заявки, но и повишава прозрачността и общата ефективност на бизнес управлението.

III. Основни предизвикателства пред управлението на Rent-a-car услуги

1. **Сложност на администрирането**
   Всяка Rent-a-car компания е изправена пред предизвикателството да следи множество променливи едновременно: наличност на превозни средства, състояние на автопарка и заетост на шофьорите. Грешките в графика или дублирането на резервации могат да доведат до сериозни финансови загуби и увреждане на имиджа на бранда.

2. **Трудност при съхраняването и управлението на данни**

   Често информацията за клиентите и историята на наемите е разпръсната в различни формати или отделни софтуерни инструменти, което забавя работата и води до несъответствия. **PremiumCarRentX** решава този проблем чрез централизация на всички данни в единна, сигурна база данни.

3. **Липса на интеграция и автоматизация**

   Използването на отделни инструменти за проверка на наличност и за финансови изчисления създава предпоставки за неефективност. Платформата предлага обединена система, която автоматизира изчисляването на наемните цени и синхронизира графиците на автомобилите и шофьорите в реално време.

IV. Как PremiumCarRentX помага за решаване на тези проблеми?

1. **Централизирано управление** – Всички данни за автопарка, професионалните шофьори и текущите наеми са съхранени на едно място, което улеснява контрола и достъпа до информация.

2. **Гъвкавост и луксозен дизайн** – Интерфейсът на системата е проектиран с внимание към детайла (Glassmorphism дизайн), осигурявайки лесна навигация и бърза адаптация на потребителите към софтуера.

V. Значение на дигитализацията в съвременния бизнес

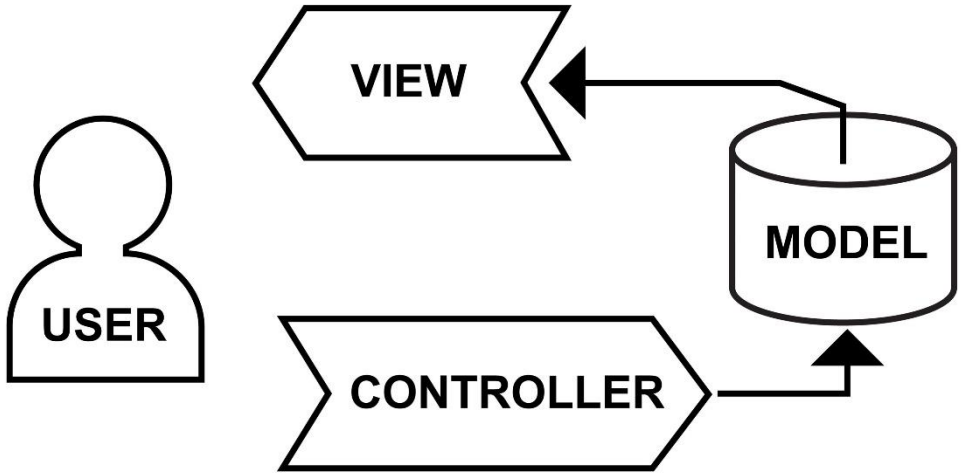
Дигитализацията е движеща сила в модерния транспортен сектор. Софтуерни решения като **PremiumCarRentX** дават възможност на компаниите да използват съвременни аналитични инструменти, да подобрят координацията между отделите и да оптимизират ресурсите си. Използването на специализирана софтуерна платформа вече не е просто предимство, а абсолютна необходимост за всеки бизнес, който иска да бъде конкурентоспособен на глобалния пазар.

MVC (Model-View-Controller)

I. ПРЕДМЕТ И ЗАДАЧИ НА MVC

Архитектурният модел MVC (Model-View-Controller) е фундаментална концепция в съвременното софтуерно инженерство, върху която е изградено приложението **PremiumCarRentX**. Този подход разделя софтуера на три взаимосвързани компонента, което позволява паралелна разработка, по-лесна поддръжка и висока степен на организираност на програмния код.

II. КОМПОНЕНТИ НА MVC



MODEL - VIEW - CONTROLLER PATTERN

1. **Модел (Model)**

   Моделът е сърцето на приложението, отговарящо за данните и бизнес логиката. В **PremiumCarRentX** моделът управлява информацията за автомобилите, шофьорите и наемите. Той извършва операции като:

   - Извличане на списъци с автомобили и техните характеристики от базата данни.
   - Валидиране на входните данни (например проверка дали датата на връщане е след датата на наемане).
   - Изчисляване на крайната цена на наема, включваща дневната тарифа и таксата за шофьор.

2. **Изглед (View)**

   Изгледът е визуалният интерфейс, с който потребителят взаимодейства. Той е изцяло фокусиран върху представянето на информацията и не съдържа логика за обработка. Неговите задачи включват:

   - Визуализиране на каталога с автомобили и профилите на шофьорите чрез Razor страници.
   - Показване на форми за вход, регистрация и резервация.
   - Изпращане на заявки към контролера при натискане на бутони (например „Резервирай сега“).

3. **Контролер (Controller)**

   Контролерът действа като координатор между Модела и Изгледа. В проекта контролери като CarsController и DriversController приемат

потребителските заявки и определят как да се обработят те. Отговорностите му включват:

- Получаване на заявки за филтриране на автомобили по цена или клас.
- Извикване на логиката за запис на нова резервация в базата данни.
- Обновяване на изгледа (View) с резултатите от търсенето.

Примерен Работен Поток при резервация:

1. Потребителят избира автомобил и натиска бутона за детайли.
2. Изгледът (View) изпраща ID на колата към CarsController.
3. Контролерът изисква данните за този автомобил от Модела (Model).
4. Моделът извлича информацията от SQL базата данни.
5. Контролерът предава получените данни към страницата с детайли.
6. Изгледът визуализира снимката и характеристиките на колата за потребителя.

III. ПРЕДИМСТВА НА MVC АРХИТЕКТУРАТА

- **Разделяне на отговорностите:** Позволява дизайнерът да работи върху CSS стиловете и "Glassmorphism" ефектите, без да засяга логиката за изчисление на цените в модела.
- **Лесна поддръжка:** Може да се промени структурата на формата за резервация, без да се налага промяна в начина, по който базата данни съхранява информацията.
- **Мащабируемост:** Платформата може лесно да бъде разширена с нови модули (например за застраховки), без да се нарушава стабилността на текущите функции.

IV. ПРИЛОЖЕНИЯ НА MVC

- **Уеб приложения:** Най-честото приложение, реализирано в този проект чрез фреймуърка **ASP.NET Core MVC**, който предоставя мощни инструменти за разработка на сигурни и бързи уеб платформи.

- **Мобилни приложения:** Принципите на MVC се използват и при разработката на мобилни версии за Android и iOS, което улеснява бъдещото разширение на **PremiumCarRentX** към мобилните пазари.

Основна част (Структура на проекта)

I. Слой за база данни (Data Layer)

Data layer в контекста на архитектурния модел MVC (Model-View-Controller) представлява частта от приложението **PremiumCarRentX**, която

отговаря за взаимодействието с базата данни или други източници на данни. Този слой играе ключова роля в управлението и обработката на информацията за автомобили, шофьори и наеми, като се грижи за тяхното съхранение, извличане и актуализиране. В MVC архитектурата данните и логиката на приложението се разделят по слоеве, което прави кода по-малко свързан и по-лесен за поддръжка.

Основни характеристики на Data layer в PremiumCarRentX:

1. **Model (Модел):** Data layer е основна част от Model слоя в MVC, който се грижи за бизнес логиката и данните.
2. **Абстракция на данни:** Предоставя слой, благодарение на който останалата част от приложението (контролери и изгледи) не се нуждае от детайлни познания за начина на съхранение на данните.
3. **Работа с база данни:** Този слой извършва CRUD операции (Create, Read, Update, Delete) върху SQL базата данни.
4. **ORM (Entity Framework Core):** Използва се библиотеката Entity Framework Core за .NET, която улеснява работата с базата данни чрез преобразуване на C# моделите в релационни таблици.
5. **Сигурност и валидация на данни:** Извършва се валидация на данните (например цени и дати), преди те да бъдат трайно записани в базата.

1. **Entities (Примерът с клас Car)**

В контекста на Entity Framework, **Entity (същност)** представлява клас, свързан с таблица в базата данни. Той описва структурата на автомобила и включва свойства, които съответстват на колоните в таблицата.

```
public class Car : BaseEntity
{
    public string Brand { get; set; }        // Марка на автомобила
    public string Model { get; set; }        // Модел на автомобила
    public int Year { get; set; }            // Година на производство
    public string Class { get; set; }        // Клас (напр. Луксозен, Ежедневен)
    public string Transmission { get; set; } // Скоростна кутия
```

```
    public decimal PricePerDay { get; set; } // Цена за наем на ден
    public string ImageUrl { get; set; }     // Линк към снимка
    public bool IsAvailable { get; set; }    // Наличност на автомобила
    public virtual List<Rental> Rentals { get; set; } // Списък с резервациите на колата
}
```

В този клас:

- Свойствата като **Brand**, **Model**, **Year** и **PricePerDay** са основните характеристики на автомобила.
- **Rentals** е навигационно свойство, което показва връзката с таблицата за наеми (резервации).
- Наследяването от **BaseEntity** осигурява общи свойства като уникален **Id**, който е необходим за идентификация на всяка същност.

2. Миграции в Entity Framework

Миграциите в Entity Framework Core се използват за управление на промените в базата данни на **PremiumCarRentX**. Те позволяват на разработчика да актуализира схемата на базата данни по безопасен начин, без да губи съществуващите данни за автомобили и резервации. Това е процесът, чрез който се генерират SQL скриптове, променящи структурата на таблиците според промените в C# моделите (същностите).

Когато добавиш ново свойство в класа Car или създадеш новата същност Rental, използваш миграции, за да отразиш тези промени в SQL Server.

Пример за използване на миграции в проекта:

1. За създаване на нова миграция:

   dotnet ef migrations add InitialCreate

2. За прилагане на миграцията към базата данни:
   dotnet ef database update

Това актуализира базата данни така, че тя да отразява новата структура на класовете в проекта.

3. DbContext (Пример с ApplicationDbContext)

DbContext е основният клас в Entity Framework Core, който управлява връзката с базата данни и служи като мост за достъп до данните. Той съдържа DbSet свойства, които представляват колекции от обекти, съответстващи на таблиците в SQL базата данни.
В проекта **PremiumCarRentX**, този клас се нарича ApplicationDbContext и дефинира структурата на базата данни за управление на автомобили под наем.

1. Основни свойства (DbSet)

В контекста на приложението са дефинирани следните таблици:

- **public DbSet<Car> Cars { get; set; }**: Таблица, съхраняваща информация за всички налични автомобили (марка, модел, цена и др.).
- **public DbSet<Driver> Drivers { get; set; }**: Таблица с данни за професионалните шофьори.
- **public DbSet<Rental> Rentals { get; set; }**: Таблица, която записва всички направени резервации и свързва автомобил, шофьор и потребител.
- **public DbSet<ApplicationUser> Users { get; set; }**: Разширена таблица за потребителите на системата (чрез ASP.NET Core Identity).

2. Конструктори

Класът използва стандартни конструктори за инициализация:

- **Празен конструктор**: Използва се за вътрешни операции на рамката.
- **Конструктор с DbContextOptions**: Позволява динамичното конфигуриране на връзката с базата данни (например задаване на Connection String от appsettings.json) при стартиране на приложението.

3. Конфигуриране на контекста (OnConfiguring)

Този метод се използва за настройка на поведението на контекста. В проекта може да бъде активирано **Lazy Loading**, което позволява свързаните данни (например резервациите към конкретна кола) да се зареждат автоматично само при поискване.

4. Конфигуриране на модела (OnModelCreating)

Този метод е от критично значение за дефиниране на връзките между таблиците и правилата за интегритет на данните.

Примерни връзки в PremiumCarRentX:

- **Връзка Car към Rentals**: Един автомобил може да бъде наеман многократно (One-to-Many). При опит за изтриване на автомобил с активна история, системата може да използва Restrict, за да предотврати загуба на данни.
- **Връзка Driver към Rentals**: Един шофьор може да участва в множество резервации.
- **Каскадно изтриване (Cascade Delete)**: Ако потребителски профил бъде премахнат, системата може автоматично да изтрие и неговите лични записи за резервации, за да поддържа базата чиста.

5. Начални данни (Data Seeding)
Чрез метода modelBuilder.Entity<User>().HasData(...) в базата данни се вмъкват първоначални данни веднага след миграцията. Това се използва най-често за:

- **Създаване на администраторски профил**: Гарантира, че при първото стартиране на системата ще има потребител с права на Admin, който да може да добавя нови автомобили и шофьори.
- **Дефиниране на роли**: Автоматично създаване на ролите Admin и User.

Класът ApplicationDbContext е централното звено на **PremiumCarRentX**, което гарантира, че данните са правилно структурирани, сигурни и лесно достъпни за контролерите на приложението.

6. **Клас DbSeeder (Начално попълване на данни)**
Класът **DbSeeder** е помощен статичен клас, чиято основна задача е автоматичното конфигуриране на началната среда на приложението **PremiumCarRentX**. Той гарантира, че при първото стартиране на системата, тя ще разполага с необходимите системни роли и администраторски профил, без да се налага ръчна намеса в базата данни.

```
using Microsoft.AspNetCore.Identity;
using Rent_a_car.Models;

namespace Rent_a_car.Data
{
    public static class DbSeeder
    {
        public static async Task SeedRolesAndAdminAsync(
            IServiceProvider serviceProvider)
        {
            var roleManager = serviceProvider
                .GetRequiredService<RoleManager<IdentityRole>>();

            var userManager = serviceProvider
                .GetRequiredService<UserManager<ApplicationUser>>();

            string[] roles = { "Admin", "Client", "Driver" };

            foreach (var role in roles)
            {
                if (!await roleManager.RoleExistsAsync(role))
                {
                    await roleManager.CreateAsync(
                        new IdentityRole(role));
                }
```

```
        }

        var adminEmail = "admin@rentacar.com";

        var adminUser = await userManager
            .FindByEmailAsync(adminEmail);

        if (adminUser == null)
        {
            var admin = new ApplicationUser
            {
                FullName = "System Administrator",
                UserName = adminEmail,
                Email = adminEmail,
                EmailConfirmed = true
            };

            var result = await userManager
                .CreateAsync(admin, "Admin123!");

            if (result.Succeeded)
            {
                await userManager
                    .AddToRoleAsync(admin, "Admin");
            }
        }
    }
  }
}
```

1. Предназначение и функционалност

Началното попълване на данни (Data Seeding) е критичен процес в съвременното софтуерно инженерство. Той позволява на приложението да бъде „готово за работа“ веднага след миграция на базата данни.

Основни задачи на класа:

- **Инициализация на роли:** Дефинира нивата на достъп в системата.
- **Създаване на главен администратор:** Гарантира наличието на потребител с пълни права за управление на автопарка и шофьорите.

2. Програмна реализация

Методът **SeedRolesAndAdminAsync** приема IServiceProvider, чрез който извлича необходимите услуги от Dependency Injection (DI) контейнера на ASP.NET Core:

- **RoleManager<IdentityRole>:** Използва се за управление на потребителските роли в Identity системата.
- **UserManager<ApplicationUser>:** Използва се за създаване и управление на потребителските профили.

3. Етапи на изпълнение

Логиката в класа е разделена на две основни фази:

А. Инициализиране на системни роли: Програмата обхожда масив от предварително дефинирани роли: **"Admin"**, **"Client"** и **"Driver"**. За всяка от тях се извършва асинхронна проверка дали вече съществува в базата данни чрез RoleExistsAsync. Ако ролята липсва, тя се създава автоматично.

Б. Създаване на системен администратор:

1. **Търсене:** Системата проверява дали съществува потребител с имейл admin@rentacar.com чрез метода FindByEmailAsync.
2. **Създаване:** Ако такъв потребител не е намерен, се инстанцира нов обект от тип ApplicationUser с потвърден имейл и системно име.
3. **Задаване на парола и права:** Чрез CreateAsync се задава сигурна парола (Admin123!). При успех, на новия потребител се присвоява ролята **"Admin"** чрез метода AddToRoleAsync.

4. Значение за сигурността

Използването на този клас предотвратява ситуации, в които административният панел на **PremiumCarRentX** би бил недостъпен поради липса на оторизиран потребител. Това е част от стратегията за управление на достъпа и ролевите политики, описани в плана за сигурност на приложението.

II. СЛОЙ ЗА УСЛУГИ (SERVICE LAYER)

Слой на услугите (Service Layer) е фундаментална част от архитектурата на **PremiumCarRentX** в рамките на MVC структурата. Неговата основна цел е да изолира сложната бизнес логика от контролерите, което осигурява по-лесна поддръжка, тестване и разширяване на приложението в бъдеще.

1. Роля на слоя на услугите

- **Абстракция на бизнес логика:** Слойят на услугите е мястото, където се съхранява цялата логика за обработка на данните за наемите. Контролерите в **PremiumCarRentX** не трябва да съдържат

тежка логика; те делегират тези задачи към услугите, което ги прави по-прости и чисти.

- **Организиране на операциите:** Този слой централизира операциите върху данни, като CRUD (създаване, четене, обновяване, изтриване) и специфични функции, свързани с наемането на автомобили. Това помага за избягване на дублирането на код.
- **Лесно разширяване и поддръжка:** Когато логиката е изнесена тук, добавянето на нови функционалности (например промоционални кодове или застраховки) става без да се засягат контролерите или изгледите.
- **Повторно използване:** Услугите могат да се извикват от различни части на приложението, спестявайки време и усилия при разработката.
- **Тестируемост:** Бизнес логиката може да бъде тествана независимо от потребителския интерфейс, което улеснява откриването на грешки чрез юнит тестове.

2. Структура на слой на услугите

В слоя на услугите на **PremiumCarRentX** се намират класове, които реализират:

- Стандартни операции за управление на автомобили, шофьори и потребители.
- Специфични за домейни операции, като автоматично изчисляване на общата сума за престой въз основа на дни и избрани екстри (напр. личен шофьор).
- Директно взаимодействие с ApplicationDbContext за комуникация с базата данни.

III. СПОДЕЛЕН ПРОЕКТ (SHARED)

В архитектурата на **PremiumCarRentX** споделеният проект играе ролята на централизирана библиотека с код, която се използва от всички останали слоеве на приложението. Това осигурява консистентност на данните и избягване на дублирането на логика. В него се съхраняват ресурси, които не са строго обвързани с презентацията или базата данни, а служат като „общ език“ за системата.

Основни ползи от споделения проект в PremiumCarRentX:

- **Централизиране на логиката:** Всички изчисления (например калкулиране на цена) и валидации се дефинират на едно място.
- **Повторна употреба:** Моделите и интерфейсите се използват както от услугите (Services), така и от контролерите.
- **Гъвкавост:** Промяната на даден параметър (например добавяне на нов тип скоростна кутия в Enum) автоматично се отразява в цялото приложение.

1. Attributes (Атрибути)

Атрибутите се използват за метаданни и автоматизация. В проекта могат да се използват атрибути за валидация или за автоматично обвързване (Binding), което гарантира, че методите работят в правилния контекст и данните от потребителските форми се обработват сигурно.

IV. ПРЕЗЕНТАЦИОНЕН СЛОЙ (PRESENTATION LAYER)

Презентационният слой в архитектурата на **PremiumCarRentX** е отговорен за взаимодействието с крайния потребител и визуалното представяне на данните. В рамките на MVC модела, този слой се реализира чрез **Изгледите (Views)** и **ViewModels**, като управлението на заявките се поема от **Контролерите (Controllers)**.

1. Основни функции на презентационния слой в PremiumCarRentX:

- **Изображение на данни (View):** Представя информацията за наличните автомобили, характеристиките им и профилите на шофьорите по удобен и луксозен начин.
- **Визуализация и оформление (Glassmorphism):** Използва се модерен UI дизайн със „стъклен ефект“ (glass-card), реализиран чрез CSS и Razor синтаксис. Това придава на приложението премиум усещане, съответстващо на бранда.
- **Интеракция с потребителя:** Слоят събира данни от потребителя чрез форми за търсене, филтриране и потвърждаване на резервации, които се изпращат към контролерите за обработка.
- **Отделеност на представянето:** Дизайнът на страниците е напълно отделен от бизнес логиката за изчисляване на цени, което позволява промени по интерфейса без риск за функционалността на системата.

2. Controllers (Пример с CarsController)

Контролерът е „мозъкът“ на презентационния слой. Той приема заявките от браузъра, комуникира с базата данни (или услугите) и връща съответния изглед.

```
[Authorize(Roles = "Admin")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
[Bind("Id,Brand,Model,Year,Class,Transmission,PricePerDay,ImageUrl,IsAvailable")] Car car)
{
    if (id != car.Id) return NotFound();

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(car);
            await _context.SaveChangesAsync();
            TempData["SuccessMessage"] = "Автомобилът беше редактиран успешно.";
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!CarExists(car.Id)) return NotFound();
            else throw;
        }
    }
    return View(car);
}
```

Обяснение на логиката:

- **Атрибути:** [Authorize(Roles = "Admin")] гарантира, че само администратори могат да променят автопарка. [ValidateAntiForgeryToken] защитава от хакерски атаки.
- **ModelState.IsValid:** Проверява дали данните от формата (например цената или годината) отговарят на изискванията, преди да ги запише в базата.
- **TempData:** Използва се за предаване на съобщения за успех към изгледа след пренасочване.

---

3. ViewModels (Пример с RentalCreateVM)

ViewModels се използват за пренасяне на специфични данни, нужни само за конкретния екран, без да се разкрива цялата структура на базата данни.

```
public class RentalCreateViewModel
{
    public int CarId { get; set; }
    public string? CarName { get; set; }
    public decimal PricePerDay { get; set; }

    [Required(ErrorMessage = "Началната дата е задължителна")]
    [DataType(DataType.Date)]
    public DateTime StartDate { get; set; }

    [Required(ErrorMessage = "Крайната дата е задължителна")]
    [DataType(DataType.Date)]
    public DateTime EndDate { get; set; }

    public bool WithDriver { get; set; }

    public int? DriverId { get; set; }
    public List<SelectListItem>? Drivers { get; set; }
}
```

Data Annotations: Атрибути като [Required] и [Display] автоматизират валидацията и генерирането на етикети във формите.

4. Views (Пример с _Layout.cshtml и Каталога)

Изгледите в **PremiumCarRentX** използват Razor двигателя за динамично генериране на HTML.

_Layout.cshtml (Главен шаблон): Осигурява единна навигационна лента, лого и футър за целия сайт. В него се проверява дали потребителят е влязъл в системата (User.Identity.IsAuthenticated), за да се покажат бутоните за „Изход“ или „Моите резервации“.

Cars/Index.cshtml (Каталог): Показва списък с всички автомобили чрез цикъл @foreach. Всеки автомобил е представен в glass-card контейнер със снимка, цена и бутон „Детайли“.

---

5. Appsettings.json

Това е файлът, който съхранява конфигурацията на приложението.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=ALEXNINJA\\SQLEXPRESS;Database=Rent_a_carDb;Trusted_Connection=True;Multiple
ActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

ConnectionStrings: Сочи към SQL Server базата данни, където се съхраняват колите и резервациите.

7. Program.cs (Сърцето на конфигурацията)

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Rent_a_car.Data;
using Rent_a_car.Models;

var builder = WebApplication.CreateBuilder(args);

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddDefaultIdentity<ApplicationUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Identity/Account/Login";
    options.LogoutPath = "/Identity/Account/Logout";
    options.AccessDeniedPath = "/Identity/Account/AccessDenied";
});

builder.Services.AddControllersWithViews();
```

```
builder.Services.AddRazorPages();

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    await DbSeeder.SeedRolesAndAdminAsync(services);
}

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();

app.Run();
```

Файлът Program.cs служи като входна точка (Entry Point) за приложението PremiumCarRentX. Неговата роля е да инициализира уеб сървъра, да конфигурира необходимите услуги (Dependency Injection) и да определи как се обработват входящите HTTP заявки.

1. Конфигурация на услугите (Dependency Injection)

В първата част на файла се регистрират компонентите, които приложението използва:

База данни: Чрез AddDbContext<ApplicationDbContext> се установява връзка със SQL Server, като данните за достъп се извличат динамично от appsettings.json чрез връзката DefaultConnection.

Система за идентичност (Identity): Регистрира се ApplicationUser за управление на потребителите. Тук са дефинирани политиките за сигурност на паролите:

Минимална дължина от 6 символа.

Задължително наличие на цифра, малка и главна буква.

Конфигурирано е използването на роли (AddRoles<IdentityRole>), което позволява разделението на достъпа между администратори, клиенти и шофьори.

Бисквитки (Authentication Cookies): Дефинирани са пътищата за вход (LoginPath), изход (LogoutPath) и отказ на достъп (AccessDeniedPath), които пренасочват потребителя към съответните страници на Identity модула.

MVC и Razor Pages: Приложението е конфигурирано да поддържа както стандартни контролери с изгледи (Model-View-Controller), така и Razor Pages (за нуждите на Identity).

2. Инициализация на данни (Data Seeding)

След изграждането на обекта app, се създава временен обхват (scope), в който се извиква статичният метод DbSeeder.SeedRolesAndAdminAsync. Този процес гарантира, че:

Системните роли съществуват в базата данни.

При първоначално стартиране автоматично се създава администраторски акаунт.

3. Конвейер за обработка на заявки (Middleware Pipeline)

Тази част определя логическата последователност, през която минава всяка заявка:

HTTPS Redirection & Static Files: Гарантира се сигурна връзка и се позволява зареждането на CSS, JavaScript и изображения от папката wwwroot.

Routing: Определя се механизмът за намиране на правилния контролер.

Authentication & Authorization: Проверява се самоличността на потребителя (кой е той?) и неговите права (какво може да прави?). Важно: UseAuthentication винаги трябва да бъде преди UseAuthorization.

Endpoint Routing: Дефиниран е стандартният маршрут по подразбиране: Home контролер и Index екшън.

8. **Компонентни тестове**
   Компонентното тестване в MVC архитектурата служи като мост между изолираните модулни тестове и тежките интеграционни

сценарии, като се фокусира върху съвместната работа на група тясно свързани класове. За разлика от unit тестовете, тук често се избягва пълното имитиране (mocking) на вътрешната логика, като вместо това се използват реални услуги, валидатори и мапери, за да се гарантира, че данните преминават коректно през различните слоеве. Основната цел е да се провери бизнес потокът в рамките на един логически модул — например дали при подаване на форма контролерът успява да валидира модела, да обработи данните чрез услугата и да върне правилния изглед.

```
[TestFixture]
public class RentalsControllerTests
{
   private ApplicationDbContext _context;
   private Mock<UserManager<ApplicationUser>> _mockUserManager;
   private Mock<SignInManager<ApplicationUser>> _mockSignInManager;
   private RentalsController _controller;

   [SetUp]
   public void Setup()
   {
      // 1. Настройка на InMemory Database
      var options = new DbContextOptionsBuilder<ApplicationDbContext>()
         .UseInMemoryDatabase(databaseName: "RentalsTestDb_" +
Guid.NewGuid().ToString())
         .Options;
      _context = new ApplicationDbContext(options);

      // 2. Mock-ване на UserManager
      var userStore = new Mock<IUserStore<ApplicationUser>>();
      _mockUserManager = new Mock<UserManager<ApplicationUser>>(userStore.Object,
null, null, null, null, null, null, null, null);

      // 3. Mock-ване на SignInManager
      var contextAccessor = new Mock<IHttpContextAccessor>();
      var claimsFactory = new Mock<IUserClaimsPrincipalFactory<ApplicationUser>>();
      _mockSignInManager = new Mock<SignInManager<ApplicationUser>>(
         _mockUserManager.Object, contextAccessor.Object, claimsFactory.Object, null, null,
null, null);

      // 4. Инициализация на контролера
      _controller = new RentalsController(_context, _mockUserManager.Object,
_mockSignInManager.Object);

      // 5. Настройка на TempData
      var tempDataProvider = new Mock<ITempDataProvider>();
      _controller.TempData = new TempDataDictionary(new DefaultHttpContext(),
tempDataProvider.Object);
```

```
}
```

1. Изолация чрез InMemory база данни

Вместо да се свързваш с истинския SQL Server (което е бавно и може да промени реални данни), ти създаваш **база данни в оперативната памет (RAM)**.

- Командата Guid.NewGuid().ToString() гарантира, че всеки отделен тест получава чисто нова, празна база данни, за да не си пречат един на друг.

2. Симулиране на сложни зависимости (Mocking)

Контролерът RentalsController изисква UserManager и SignInManager, за да работи с потребители. Тези компоненти са много сложни и имат десетки зависимости.

- Чрез библиотеката **Moq** ти създаваш „кухи“ версии (Mocks) на тези мениджъри. Те изглеждат като истински за контролера, но не изискват реална логика за сигурност или връзка с интернет.

3. Ръчно инжектиране на зависимости (Dependency Injection)

В реалното приложение ASP.NET Core само „налива“ нужните обекти в контролера. В тестовете ти правиш това ръчно: _controller = new RentalsController(_context, _mockUserManager.Object, _mockSignInManager.Object); Тук подаваш своята фалшива база данни и фалшивите мениджъри в конструктора на контролера.

4. Поддръжка на TempData

Твоят контролер използва TempData, за да предава съобщения (например: „Резервацията е успешна!“).

- По подразбиране в тестовете TempData е null и би довело до грешка. Последният блок от кода създава „празно чекмедже“ за тези съобщения, за да може тестът да мине успешно.

Програмна реализация

V. ИЗПОЛЗВАНИ ТЕХНОЛОГИИ

При създаването на електронното приложение **PremiumCarRentX** са използвани съвременни и широко разпространени технологии, които позволяват изграждането на динамична, сигурна и визуално впечатляваща уеб платформа за наемане на автомобили. Комбинацията от front-end и back-end технологии осигурява пълна функционалност на системата – от представянето на луксозния автопарк пред потребителя до сложната обработка на резервациите и съхранението на личните данни в сървъра.

1. HTML

HTML (HyperText Markup Language) е основният език за структуриране на уеб съдържанието в проекта. С него се изгражда „скелетът“ на всички страници – от началната страница с акценти до подробните форми за наемане на конкретен автомобил. Всеки елемент в **PremiumCarRentX** – бутони за резервация, полета за избор на дати, заглавия и спецификации на колите – е описан чрез HTML тагове.

Използвана е най-новата версия – **HTML5**, която позволява интеграция на мултимедия (снимки на висока резолюция) и семантични тагове, които подобряват четимостта на кода и оптимизацията за търсачки. HTML служи като здрава основа, върху която се наслагват стиловете и динамиката на приложението.

2. CSS

CSS (Cascading Style Sheets) е езикът, отговорен за визуалната идентичност на **PremiumCarRentX**. В този проект CSS играе ключова роля, тъй като чрез него е постигнат специфичният **„Premium“ вид** на сайта. Използвани са съвременни техники като **Glassmorphism** (ефект на матирано стъкло), полупрозрачни цветове, сенки и градиенти, които придават луксозно усещане на интерфейса.

С помощта на CSS е реализиран и адаптивен (**responsive**) дизайн. Благодарение на него потребителите могат лесно да разглеждат каталога и да правят резервации от всякакви устройства – смартфони, таблети и настолни компютри, като навигацията се пренастройва автоматично според ширината на екрана.

3. JavaScript

JavaScript е езикът, използван за добавяне на интелигентно поведение и интерактивност в уеб страниците на приложението. В **PremiumCarRentX** той се прилага за подобряване на потребителското изживяване чрез функции като:

- Автоматично изчисляване на общата сума за наем в реално време според броя на избраните дни.
- Валидиране на формите за резервация още преди те да бъдат изпратени към сървъра.
- Създаване на динамични ефекти при превъртане и преходи между различните секции на сайта.

JavaScript позволява промяна на съдържанието в реално време, без да се налага досадно презареждане на цялата страница при всяко действие.

4. SQL

SQL (Structured Query Language) е езикът, чрез който приложението комуникира със своята база данни. В контекста на **PremiumCarRentX**, SQL се използва за управление на релационните връзки между основните обекти: Автомобили, Шофьори, Потребители и Резервации.

Чрез него се изпълняват критични операции като:

- Извличане на свободните за определен период автомобили.
- Записване на нови резервации и актуализиране на статуса на колите.
- Проверка на администраторски права при вход в системата. Най-често използваните команди са SELECT за филтриране на автопарка и UPDATE за управление на наличностите.

5. SSMS

SSMS (SQL Server Management Studio) е работната среда, използвана за администрация на базата данни на проекта. Чрез този инструмент на Microsoft е проектирана схемата на базата данни, създадени са таблиците и са конфигурирани връзките между тях (Foreign Keys).

Интегрираната среда позволява на разработчика да анализира производителността на заявките, да прави резервни копия на информацията и да извършва директни корекции по данните в таблиците, осигурявайки стабилна и надеждна работа на цялата система **PremiumCarRentX**.

Заключение

PremiumCarRentX е иновативно софтуерно решение, което отговаря на високите изисквания на съвременния бизнес за отдаване на автомобили под наем, като предлага цялостна и интегрирана платформа за управление на всички ключови аспекти на дейността. Чрез централизацията на данните за автопарка и автоматизацията на процесите по резервация, приложението значително намалява административната тежест и повишава оперативната ефективност. Платформата не само улеснява управлението на автомобилите, профилите на шофьорите и графиците на наемите, но също така осигурява бърз достъп до критична информация и позволява отлична координация между екипа и клиентите.

Дигитализацията в Rent-a-car сектора става все по-решаваща, а **PremiumCarRentX** предоставя необходимите инструменти за успешното внедряване на съвременните технологии в управлението на транспорта. В съвременния свят на бързо променящи се условия и засилена конкуренция, използването на специализирани решения като това не само оптимизира ежедневните операции, но и позволява на компанията да бъде конкурентоспособна на най-високо ниво, предлагайки „Premium“ услуга на своите потребители.

Независимо дали се използва от бутикова агенция или от компания с голям автопарк, **PremiumCarRentX** е ценен ресурс, който подобрява както текущата ефективност, така и дългосрочното стратегическо планиране, осигурявайки стабилна основа за развитие в съвременния цифров свят.

I. Начини за подобрение на PremiumCarRentX

1. **Интеграция с външни платформи и технологии** Възможността за свързване с популярни системи като Google Maps за GPS проследяване на автомобилите в реално време или интеграция с външни платформи за резервации (като Booking или Expedia) би увеличила значително обхвата на приложението. Интеграцията с финансови инструменти за автоматично издаване на фактури също би била голямо предимство.

2. **Подобрена аналитика и Изкуствен интелект (AI)** Внедряването на AI би помогнало за анализ на търсенето и автоматично предлагане на „динамично ценообразуване“ (по-високи цени през активния сезон и

отстъпки в слабия период). Също така, алгоритми могат да предсказват кога даден автомобил има нужда от техническо обслужване въз основа на изминатите километри.

3. **Мобилно приложение за клиенти и шофьори** Разработването на мобилна версия ще улесни клиентите да резервират автомобили „в движение“, а шофьорите – да получават известия за нови курсове и да управляват своя график директно от телефоните си.
4. **Разширена система за уведомления** Внедряване на автоматични SMS или имейл напомняния за край на срока на наема, предстоящи резервации или просрочени плащания. Уведомленията могат да включват и промоционални кодове за лоялни клиенти.
5. **Персонализирани пакети за наемане** Добавяне на функции за избор на допълнителни екстри в реално време – различни нива на застраховка, детски седалки, ски багажници или специфично оборудване, което автоматично да обновява крайната цена.
6. **Многоезичност и валутен конвертор** Тъй като Rent-a-car услугите често се ползват от чужденци, добавянето на многоезичен интерфейс и възможност за плащане в различни валути (EUR, USD, GBP) ще направи платформата достъпна на международно ниво.
7. **Финансов модул за проследяване на разходите** Включване на разширена система за управление на разходите по автопарка – следене на разходи за гориво, ремонти, застраховки (Гражданска отговорност, Каско) и винетки, което ще даде пълна представа за рентабилността на всеки автомобил.
8. **Система за лоялност и CRM** Интеграция на клиентски досиета, които съхраняват историята на наемите и предпочитанията на потребителите. Това ще позволи създаването на бонус системи – например „всеки 5-ти наем с 20% отстъпка“.
9. **Гъвкаво генериране на отчети** Възможност за администраторите да генерират подробни доклади за заетостта на автопарка, най-предпочитаните модели автомобили и общите приходи за определен период, които да се експортират в PDF или Excel формат.
10. **Обратна връзка и рейтинг система** Внедряването на модул, в който клиентите да оставят оценки и коментари за състоянието на автомобила и качеството на услугата на шофьора. Това би помогнало за поддържане на високото „Premium“ ниво на обслужване.

С тези подобрения **PremiumCarRentX** може да се трансформира в лидер на пазара, предоставяйки най-съвременните технологични инструменти за оптимизация на бизнеса.

Придобити знания

По време на изпълнението на дипломната работа придобих редица нови знания и умения, които са от съществено значение за съвременното разработване на уеб приложения. На първо място, се задълбочих в архитектурния модел **MVC (Model-View-Controller)**, който е основата на приложението **PremiumCarRentX**. Научих как правилно да структурирам софтуерен проект, така че бизнес логиката, потребителският интерфейс и данните да бъдат ясно разделени, което прави системата лесна за поддръжка и бъдещо надграждане.

Освен това, придобих ценен практически опит в работата с релационни бази данни – проектиране на таблици, дефиниране на сложни връзки между тях и ефективно управление на информацията чрез **Entity Framework Core**. Разширих знанията си по **ASP.NET Core**, включително работа с контролери, изграждане на динамични изгледи чрез Razor двигателя и имплементиране на надеждна логика за **CRUD операции** (създаване, четене, редактиране и изтриване) на превозни средства и потребителски записи.

Също така се научих как да внедрявам функционалности, изцяло ориентирани към реалните нужди на бизнеса и клиентите, като управление на луксозен автопарк, профили на професионални шофьори и

автоматизирана система за резервации. Това включваше не само техническа реализация, но и детайлен анализ на потребителските сценарии, прилагане на модерни **UX/UI практики** (като използването на Glassmorphism дизайн) и изграждане на математически модели за изчисляване на цени в реално време.

Придобих и важни знания относно сигурността и ролевия достъп, като внедрих **ASP.NET Core Identity** за управление на администраторски права и контрол върху действията на потребителите в системата. Осъзнах колко е важно едно приложение не само да работи коректно, но и да гарантира защитата на личните данни, да бъде достъпно и мащабируемо при нарастващ брой наеми.

Накрая, процесът на разработка ми помогна да усъвършенствам уменията си за планиране, самостоятелна работа и критично мислене при решаване на сложни програмни казуси и дебъгване на грешки в кода.

Създаването на иновативна и реалистична система като **PremiumCarRentX** затвърди моите теоретични познания и ме подготви за реалните предизвикателства, пред които се изправят специалистите в сферата на софтуерното инженерство.

Използвана литература

VI. ИЗПОЛЗВАНИ ИЗТОЧНИЦИ

При разработката на проекта **PremiumCarRentX** са използвани редица авторитетни ресурси, документация и съвременни инструменти за изкуствен интелект, които осигуриха необходимата теоретична и практическа основа за изграждането на приложението.

1. Microsoft Docs – ASP.NET Core MVC

- **Линк:** https://docs.microsoft.com/en-us/aspnet/core/mvc/

- **Описание:** Официалната документация на Microsoft предостави основните насоки за структуриране на проекта, внедряване на ASP.NET Core Identity за сигурност и конфигуриране на връзката с SQL Server чрез Entity Framework Core.

2. W3Schools – ASP.NET MVC Tutorial

- **Линк:** https://www.w3schools.com/aspnet/aspnet_mvc_intro.asp
- **Описание:** Този ресурс беше използван за бързи справки относно синтаксиса на Razor страниците, работата с HTML помощници (Helpers) и интегрирането на Bootstrap за постигане на адаптивен дизайн.

3. Gemini (Google)

- **Линк:** https://gemini.google.com
- **Описание:** Моделът с изкуствен интелект на Google изигра централна роля като технически консултант по време на разработката. **Gemini** беше използван за оптимизиране на сложни C# алгоритми, генериране на специфични CSS стилове за постигане на „Premium“ дизайна (Glassmorphism), дебъгване на грешки в базата данни и правилното структуриране на проектната документация. AI технологията спомогна за внедряването на иновативни решения и подобряване на цялостната архитектура на приложението.

4. DotNetFiddle

- **Линк:** https://dotnetfiddle.net/
- **Описание:** Онлайн платформата беше използвана за изолирано тестване на C# логика, включително математическите модели за изчисляване на наемната цена в реално време.

5. CodeProject – ASP.NET MVC Tutorials

- **Линк:** https://www.codeproject.com/Articles/25829/Introduction-to-MVC-ASP-NET
- **Описание:** Статиите в CodeProject помогнаха за разбирането на добри практики при работа с контролери и изгледи в реални бизнес сценарии.

6. Stack Overflow

- **Линк:** https://stackoverflow.com/

- **Описание:** Глобалната общност за разработчици беше незаменим ресурс за решаване на специфични технически казуси, свързани с миграциите на Entity Framework и обработката на POST заявките.

План за управление на риска

В процеса на разработка и внедряване на системата **PremiumCarRentX** е необходимо да се идентифицират и анализират възможните рискове, които могат да възпрепятстват нормалното функциониране на платформата. С цел минимизиране на въздействието от подобни ситуации са дефинирани няколко ключови риска и подходящи стратегии за тяхното управление.

Един от основните рискове е **загубата на база данни**, която може да има критично въздействие върху дейността на компанията за отдаване на автомобили под наем. За намаляване на този риск се прилагат редовни архивирания, както и използване на надеждни механизми за миграции и възстановяване чрез **Entity Framework Core**.

Друг значим риск е **несъвместимостта с бъдещи версии на .NET**, което може да доведе до загуба на функционалност или невъзможност за поддръжка на приложението в дългосрочен план. За справяне с това се планира периодична актуализация на проекта и поддържане на адекватна техническа документация.

Възможно е да се появи и **прекъсване на достъпа до външни API услуги**, като например услуги за съхранение на изображения или геолокация на офисите. За целта се прилагат стратегии като локално кеширане на данни и готовност за преминаване към алтернативни доставчици на съдържание.

Приемането на невалидни потребителски данни също представлява риск със средно въздействие върху стабилността на системата. Той се управлява чрез стриктна валидация на данните (ценови формати, дати на наемане, наличност) както на клиентско, така и на сървърно ниво, което гарантира целостта и надеждността на информацията.

Неправилното използване на правата за достъп е още един потенциално сериозен проблем, предвид факта, че системата работи с лични данни и

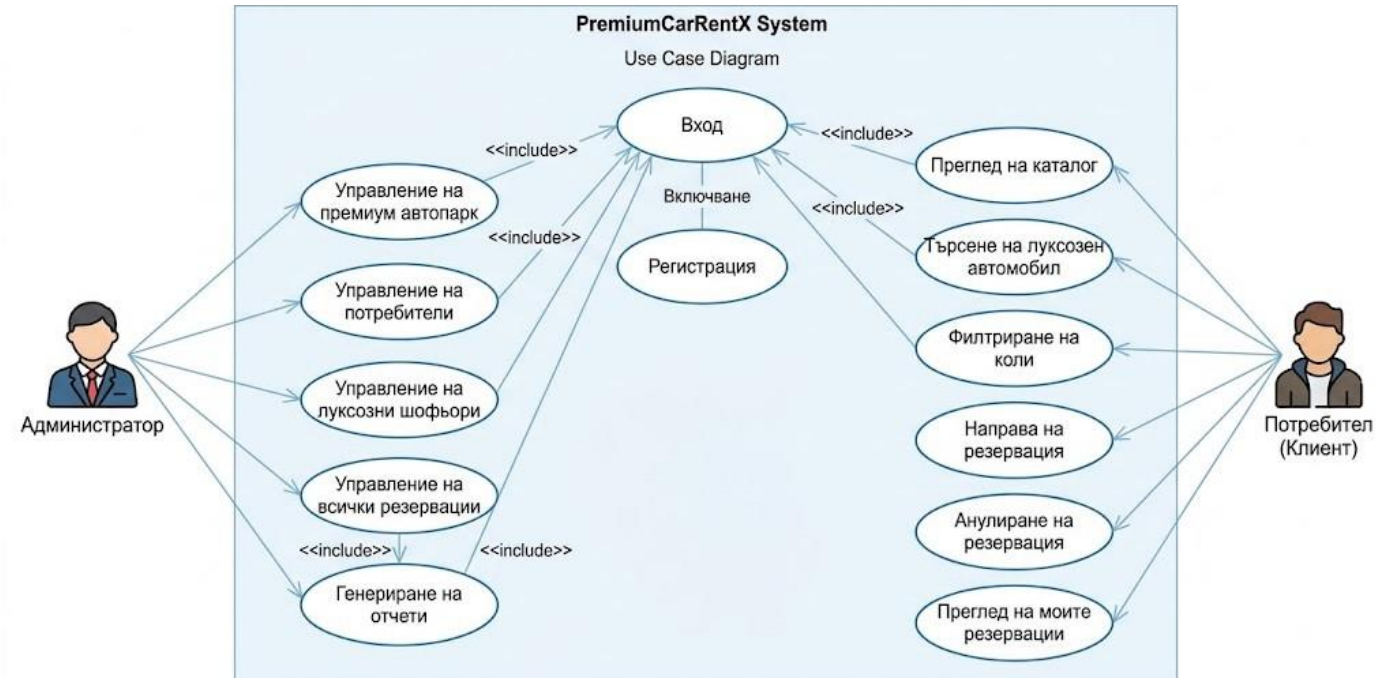
ценообразуване. За справяне с него се прилагат ясно дефинирани ролеви политики (Admin и User), както и проверки за права при всеки опит за достъп до административния панел или редактиране на автомобили.

Последният идентифициран риск е свързан с **натоварване на сървъра** при голям брой заявки (например по време на активния туристически сезон), което може да доведе до забавяне или дори срив на системата. Решенията тук включват оптимизация на SQL заявките, използване на индекси в базата данни и асинхронно зареждане на ресурсите.

Риск	Стратегия за управление
Загуба на данни	Редовни архивирания и механизми за възстановяване чрез EF Core.
Несъвместимост с .NET версии	Периодични актуализации на програмната рамка и документацията.
Прекъсване на API услуги	Локално кеширане на ресурси и използване на алтернативни доставчици.
Невалидни потребителски данни	Двустепенна валидация на данни (клиентска и сървърна).
Неправилно използване на правата	Стриктни ролеви политики (Identity) и проверки за оторизация.
Натоварване на сървъра	Оптимизация на заявките и използване на индекси в SQL Server.

Приложения

I.Use case diagram



II. SWOT Таблица

SWOT Анализ	Силни страни
Продукт	- Централизирано управление на автомобили, шофьори и резервации.

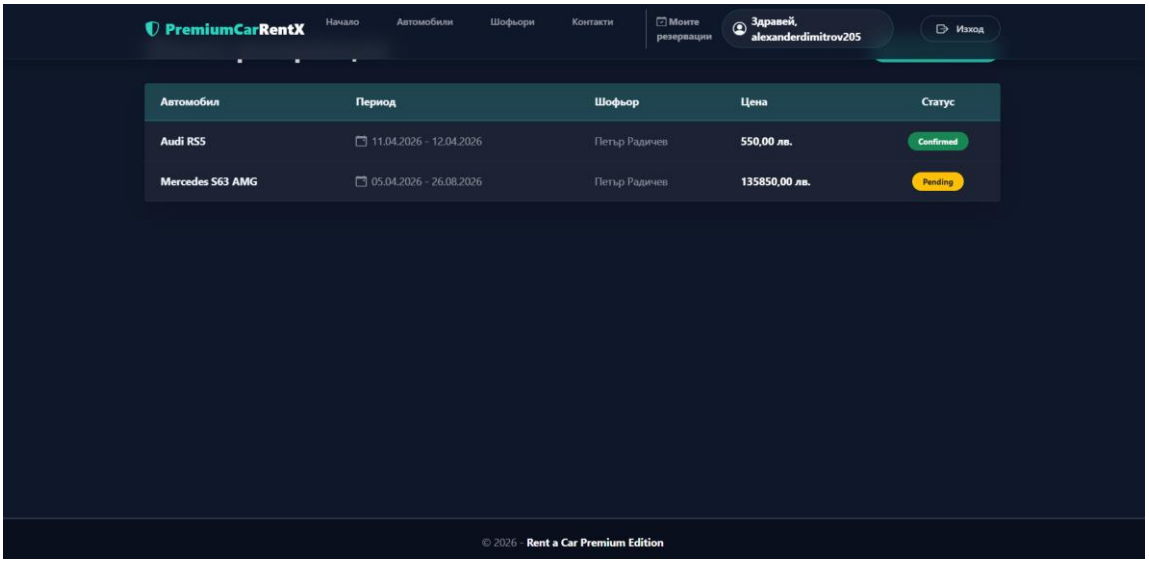
	- Автоматизирано изчисляване на цени в реално време.
Технологии	- Използване на **Entity Framework Core** за гарантиране интегритета на данните. - Модерен и луксозен **Glassmorphism** дизайн.
Възможности	Заплахи
- Интеграция с GPS системи за проследяване на автомобилите.	- Загуба на данни при неправилно конфигурирано архивиране на SQL базата.
- Внедряване на разплащателни системи (Stripe/PayPal).	- Прекъсвания в достъпа до външни Image услуги.
- Разширена аналитика с AI за прогнозиране на заетостта на автопарка.	- Прекомерно натоварване на сървъра по време на активни туристически сезони.
- Многоезична поддръжка за международни клиенти.	

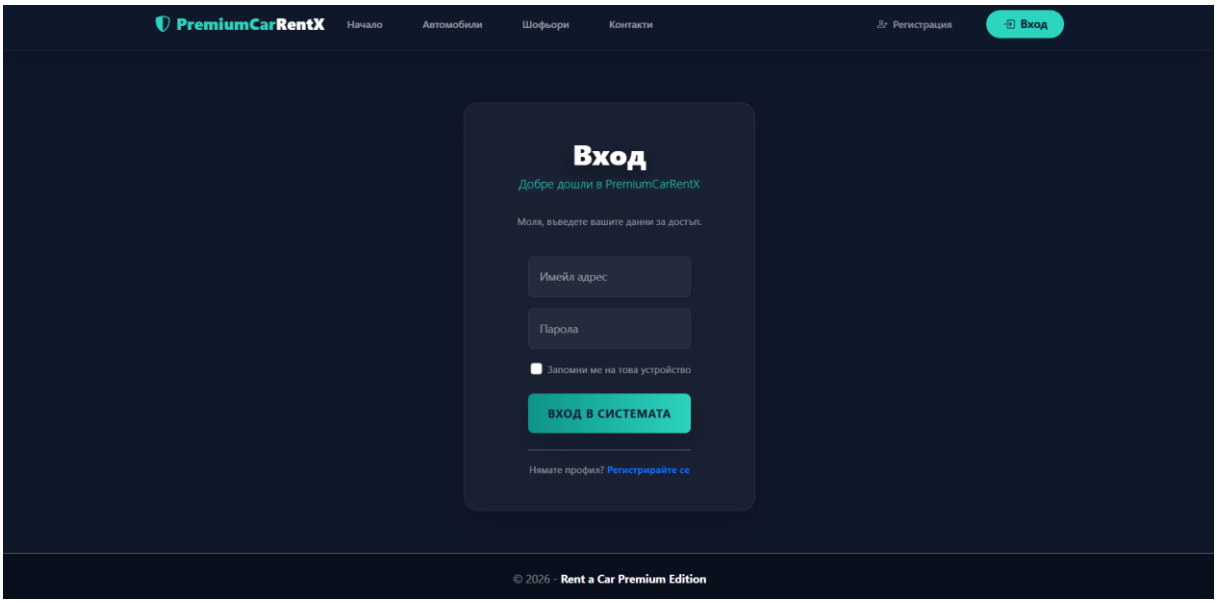
Обяснение на SWOT анализа за PremiumCarRentX:

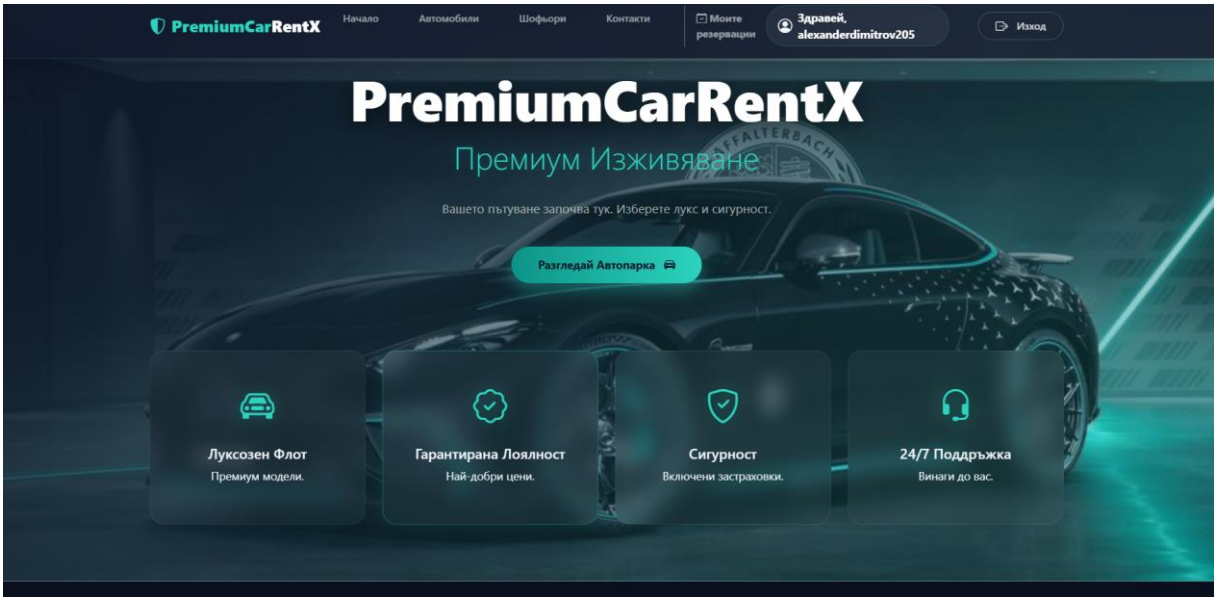
1. **Силни страни:** Проектът ти се отличава с автоматизация (не смяташ цената на ръка) и луксозна визия, която е критична за "Premium" бранда.

2. **Слаби страни:** Основната слабост е, че ако снимките ти са линкове от интернет, при изтриване на оригиналния файл, снимката в твоя сайт също изчезва.
3. **Възможности:** Тук описваш как проектът може да се превърне в истински бизнес чрез добавяне на плащания и мобилно приложение.
4. **Заплахи:** Това са външните фактори – като например сървърът да "падне", ако твърде много хора се опитват да наемат коли едновременно през лятото.

III.Работещ проект




PremiumCarRentX
Начало
Автомобили
Шофьори
Контакти
Регистрация
Вход
Вход
Добре дошли в PremiumCarRentX
Моля, въведете вашите данни за достъп.
Имейл адрес
Парола
Запомни ме на това устройство
ВХОД В СИСТЕМАТА
Нямате профил? Регистрирайте се
© 2026 - Rent a Car Premium Edition


PremiumCarRentX
Начало
Автомобили
Шофьори
Контакти
Моите резервации
Здравей, alexanderdimitrov205
Изход
PremiumCarRentX
Премиум Изживяване
Вашето пътуване започва тук. Изберете лукс и сигурност.
Разгледай Автопарка
Луксозен Флот
Премиум модели.
Гарантирана Лоялност
Най-добри цени.
Сигурност
Включени застраховки.
24/7 Поддръжка
Винаги до вас.